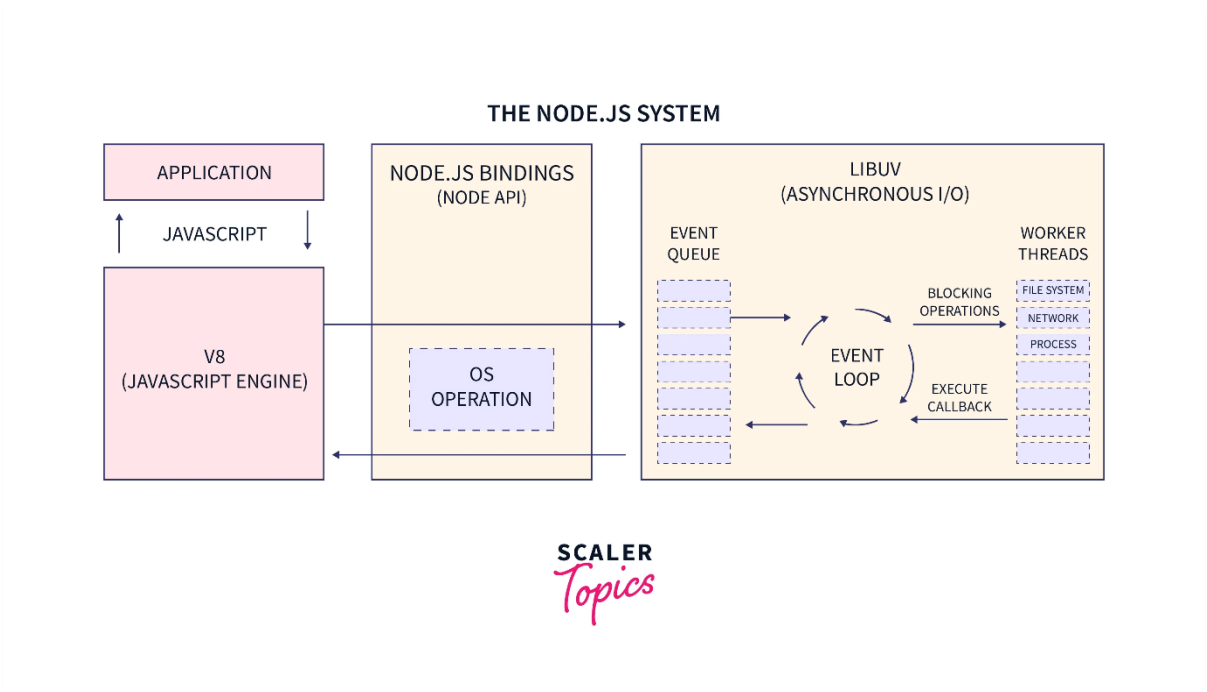
Node.js

Node.js is a JavaScript runtime built on **Google Chrome V8** engine that allows you to run JavaScript outside the browser.

Its architecture is designed for:

- High performance
- Handling thousands of concurrent connections
- Non-blocking I/O
- Event-driven programming



🔹 Core Components of Node.js Architecture

1️⃣ V8 Engine

Converts JavaScript → Machine Code

Executes JS extremely fast

Handles memory allocation and garbage collection

V8 does **NOT** handle async operations like file system or network.

2️⃣ Single-Threaded Event Loop

Node.js runs on a **single main thread**.

But here's the key:

It uses an **event loop** to handle concurrency instead of multiple threads.

This avoids:

- Thread creation overhead
- Context switching cost
- Memory duplication

🔄 How Event Loop Works

1. Execute synchronous code
2. Offload async tasks (I/O, DB, timers)
3. When async task finishes → push callback into queue
4. Event loop picks callback and executes it

3️⃣ libuv (The Real Hero)

Node uses **libuv**, a C library, to handle:

- File system
- Networking
- DNS
- Timers
- Thread pool

Without libuv, Node would be single-threaded and slow.

🔹 Event Loop Phases (Very Important)

This is where most devs get confused.

The event loop has **6 phases**:

Phase	What Happens
1️⃣ Timers	Executes setTimeout, setInterval
2️⃣ Pending Callbacks	Executes I/O callbacks
3️⃣ Idle/Prepare	Internal use
4️⃣ Poll	Retrieve new I/O events
5️⃣ Check	Executes setImmediate()
6️⃣ Close	Close events like socket.on('close')

🔁 What Is the Event Loop?

It is an infinite loop that:

1. Checks if call stack is empty
2. Pulls tasks from queues
3. Executes them
4. Repeats forever

But it's not just “one queue”.

It has **6 phases**, and each phase has its own FIFO queue.

1️⃣ Timers Phase

Executes:

- setTimeout()
- setInterval()

⚠ Important:

Timers are **not guaranteed** to run exactly at delay time.

They run:

“After minimum delay AND when the event loop reaches the timers phase.”

This does NOT mean “run immediately”.

It means:

Run when timers phase is reached and delay has expired.

2️⃣ Pending Callbacks Phase

Executes I/O callbacks that were deferred.

Example:

- Some TCP errors
- Certain system-level operations

This is not common in normal app code but important internally.

3️⃣ Idle / Prepare Phase

Internal use only.

You cannot hook into it.

Used by:

- Node internals
- libuv preparation logic

4️⃣ Poll Phase (🔥 MOST IMPORTANT)

This is the heart of the event loop.

Poll phase does TWO things:

1. Executes I/O callbacks

2. Waits for new I/O events if queue empty

```
fs.readFile("file.txt", () => {
console.log("File read");
});
```

The callback runs in the **poll phase**.

Poll Phase Behavior Is Smart

If:

- Timers are ready → move to timers phase
- setImmediate() exists → move to check phase
- No timers → wait for I/O

This dynamic behavior is what makes Node efficient.

5️⃣ Check Phase

Executes:

setImmediate(() => {})

This always runs after poll phase.

setTimeout(0) vs setImmediate()

Order depends on context.

Inside I/O callback:

```
fs.readFile("file.txt", () => {
  setTimeout(() => console.log("timeout"), 0);
  setImmediate(() => console.log("immediate"));
});
```

Output:

immediate
timeout

Why?

Because:
Poll → Check → Timers

6️⃣ Close Callbacks Phase

Handles:

socket.on("close", () => {});

Runs when connections close abruptly.

⚡ Microtasks (VERY IMPORTANT)

Microtasks are NOT a phase.

They run:

Immediately after current operation
Before moving to next phase

Example:-

```
const fs = require("fs");

// ---- TIMERS PHASE ----
setTimeout(() => {
  console.log("1️⃣ setTimeout");
}, 0);

// ---- CHECK PHASE ----
setImmediate(() => {
  console.log("2️⃣ setImmediate");
});

// ---- MICROTASKS ----
```

```
process.nextTick(() => {
  console.log("3️⃣ nextTick");
});

Promise.resolve().then(() => {
  console.log("4️⃣ promise");
});

// ---- POLL PHASE (I/O) ----
fs.readFile(__filename, () => {
  console.log("5️⃣ file read");

  setTimeout(() => {
    console.log("6️⃣ timeout inside I/O");
  }, 0);

  setImmediate(() => {
    console.log("7️⃣ immediate inside I/O");
  });

  process.nextTick(() => {
    console.log("8️⃣ nextTick inside I/O");
  });

  Promise.resolve().then(() => {
    console.log("9️⃣ promise inside I/O");
```

```
  });
});

// ---- CLOSE PHASE ----
const stream = fs.createReadStream(__filename);
stream.on("close", () => {
  console.log("🔟 close event");
});
stream.destroy();

// ---- SYNC ----
console.log("0️⃣ sync code");
```

🟢 **STEP 1 — Run All Normal Code (Top to Bottom)**

The manager reads the file (your JS file).

It:

- Registers timers
- Registers immediates
- Registers I/O
- Registers microtasks
- Registers close event

Then runs:

0️⃣ sync code

Because sync code runs immediately.

🟢 **STEP 2 — Run Microtasks (VIP Tasks)**

After sync finishes:

First:

3️⃣ nextTick

Because process.nextTick() is highest priority.

Then:

4️⃣ promise

Because Promise queue runs after nextTick queue.

Now event loop phases start.

🟡 **PHASE 1 — Timers**

setTimeout is ready.

So:

1️⃣ setTimeout

🟡 **PHASE 2 — Pending Callbacks**

Nothing here in this example.

🟡 **PHASE 3 — Idle/Prepare**

Internal. Skip.

🟡 **PHASE 4 — Poll Phase (I/O)**

The file read finishes.

So:

5️⃣ file read

Now inside that callback we scheduled:

- timeout
- immediate

- nextTick
- promise

BUT before leaving this callback…

🟢 **Microtasks inside I/O run immediately**

8️⃣ nextTick inside I/O
9️⃣ promise inside I/O

Because microtasks ALWAYS run before moving to next phase.

🟡 **PHASE 5 — Check Phase**

Now we run setImmediate.

First:

2️⃣ setImmediate

Then:

7️⃣ immediate inside I/O

Because immediates run in check phase.

🟡 **PHASE 6 — Close Callbacks**

We destroyed the stream earlier.

So:

🔟 close event

🔁 **Next Loop Cycle → Timers Again**

Now the timeout inside I/O runs:

6️⃣ timeout inside I/O

✅ **Final Output Order (Typical)**

0️⃣ sync code
3️⃣ nextTick

4️⃣ promise
1️⃣ setTimeout
5️⃣ file read
8️⃣ nextTick inside I/O
9️⃣ promise inside I/O
2️⃣ setImmediate
7️⃣ immediate inside I/O
🔟 close event
6️⃣ timeout inside I/O

🎯 The Key Truths You Must Remember

1️⃣ Microtasks run after EVERY callback

Not just at the beginning.

2️⃣ nextTick > Promise

nextTick always wins.

3️⃣ setImmediate runs after poll phase

Which is why inside I/O it beats setTimeout.

4️⃣ Poll phase is the heart

This is where real async work finishes.

Blocking vs Non-Blocking code

🔴 Blocking Code

Blocking code **stops the event loop**.

Nothing else can run until it finishes.

Example 1 — Blocking File Read

```
const fs = require("fs");

console.log("Start");

const data = fs.readFileSync("file.txt"); // BLOCKING

console.log("File read complete");
console.log("End");
```

What happens?

1. "Start" prints
2. Node freezes while reading file
3. After file finishes → prints remaining logs

During file reading:

- ❌ No other request handled
- ❌ Server is frozen

🟢 Non-Blocking Code

Non-blocking code **delegates work** and continues.

Example 2 — Non-Blocking File Read

```
const fs = require("fs");

console.log("Start");

fs.readFile("file.txt", () => {
  console.log("File read complete");
});

console.log("End");
```

What happens?

Output:

Start
End
File read complete

Node:

1. Sends file task to OS
2. Keeps running
3. Executes callback later

Imagine:

1000 users hit your API.

🔴 **With Blocking Code**

Each request:

- Blocks the entire server
- Users wait in line

If file read takes 1 second:

- 1000 users = 1000 seconds queue

Disaster.

🟢 With Non-Blocking Code

Now:

- Node delegates all reads
- Handles other requests meanwhile
- Users don’t block each other

Huge scalability difference.

What Actually Happens Internally

Blocking Code

Call Stack Busy
Event Loop Waiting
Everything Stuck

Non-Blocking Code

Call Stack Free
Task Sent to OS / Thread Pool
Event Loop Continues
Callback Later

🧩 **Types of Blocking in Node**

1️⃣ **Synchronous APIs**

- readFileSync
- writeFileSync
- execSync

2️⃣ **Heavy CPU Work**

- Encryption loops
- Image processing
- Large JSON parsing
- Big sorting algorithms

🎯 **Interview-Level Insight**

Node is not “non-blocking by default.”

It gives you:

- Non-blocking APIs
- And blocking APIs

You choose.

Worker Thread

Worker Threads solve one of Node’s biggest weaknesses:

CPU-heavy work blocks the event loop.

They allow **true parallelism** inside **Node.js**.

First: Why Worker Threads Exist

Node is:

- Single JS thread
- Event loop driven
- Great for I/O
- Bad for heavy CPU tasks

```
function heavy() {
  for (let i = 0; i < 5e9; i++) {}
}

app.get("/", (req, res) => {
  heavy();  // ❌ Blocks entire server
  res.send("Done");
});
```

While this runs:

- ❌ No other requests handled
- ❌ Server frozen

That’s where Worker Threads come in.

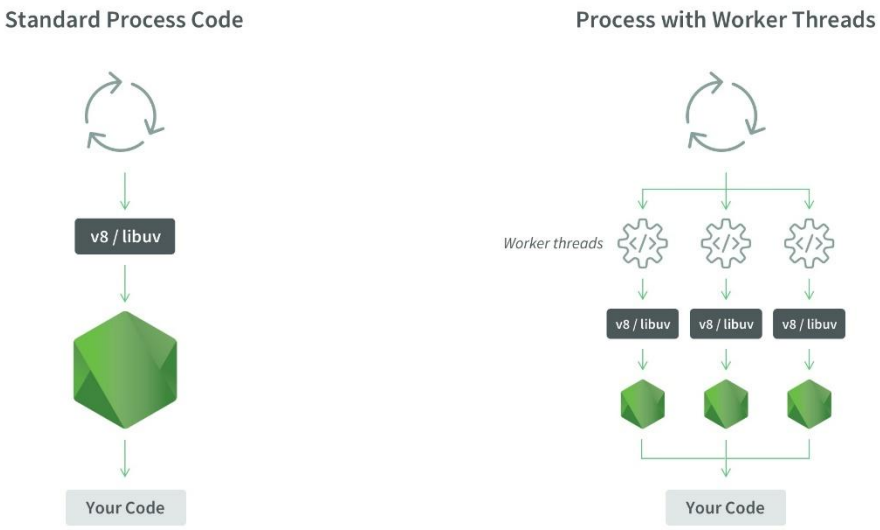
🧵 What Is a Worker Thread?

A Worker Thread:

- Is a separate JavaScript thread
- Has its own V8 instance
- Has its own event loop
- Runs in parallel

Think of it as:

Spinning up another mini-Node engine inside your app.’



How It Works Internally

Main Thread:

- Handles requests
- Runs event loop

Worker Thread:

- Runs heavy computation
- Sends result back

Communication happens via:

- Message passing
- Shared memory (optional)

📁 main.js

```
const { Worker } = require("worker_threads");

console.log("Main thread starting");

const worker = new Worker("./worker.js");

worker.on("message", (result) => {
  console.log("Result from worker:", result);
});

worker.on("error", (err) => {
  console.error(err);
});
```

```
console.log("Main thread continues...");
```

📁 **worker.js**

```
const { parentPort } = require("worker_threads");

function heavyComputation() {
  let sum = 0;
  for (let i = 0; i < 5e9; i++) {
    sum += i;
  }
  return sum;
}

const result = heavyComputation();

parentPort.postMessage(result);
```

🧠 **What Happens?**

1. Main thread starts
2. Worker thread runs heavy computation
3. Main thread does NOT block
4. Worker sends result back

Server remains responsive.

⚠ **Important Truth**

Worker Threads are NOT for:

- Database calls
- File reads
- Network calls

Because those are already non-blocking.

Worker Threads are for:

CPU-heavy synchronous JavaScript.

Use them for:

- Image processing
- Encryption
- Video encoding
- Machine learning inference
- Big data parsing
- Large JSON transforms

Do NOT use them for:

- Normal API routes
- I/O tasks
- Simple business logic

Because:
Spawning threads has overhead.

Memory leaks

A memory leak happens when:

Memory is no longer needed but is still referenced, so garbage collector cannot free it.

In simple words:

You forget to throw away things you no longer need.

Node uses **Google Chrome V8**.

V8 has:

- Stack (small, fast)
- Heap (dynamic memory)
- Garbage Collector (GC)

If no references point to an object → GC deletes it.

But if something still references it → memory stays.

Common Causes of Memory Leaks in Node

1️⃣ Global Variables (Classic Mistake)

Each request pushes data.
Never removed.

Memory grows endlessly.

2️⃣ Unremoved Event Listeners

Each request:

- Adds a new listener
- Never removes it

Eventually:
⚠ MaxListenersExceededWarning

Memory leak.

3️⃣ Closures Holding Large Objects

4️⃣ Caching Without Limits

5️⃣ setInterval Without Clearing

📊 **How to Detect Memory Leaks**

1️⃣ **Monitor Memory Usage**

console.log(process.memoryUsage());

3. Use Clinic.js (Production Tool)

   node --inspect app.js

   Open Chrome:

   chrome://inspect

🧠 Deep Insight: Why Node Is Prone to Leaks

Because:

- Long-running process
- High concurrency
- Global state is easy
- Closures are everywhere
- Caching is common

What Is an Idempotent Operation?

An operation is **idempotent** if:

Performing it multiple times has the same effect as performing it once.

In simple terms:

If you press the button 1 time or 100 times → final result is the same.

📜 Simple Real-Life Examples

- Turning a light switch **OFF** repeatedly → still OFF ✅
- Setting your profile name to "Prajwal" multiple times → still "Prajwal" ✅
- Deleting the same file twice → file remains deleted ✅

But:

- Sending ₹100 to someone twice ❌ (money deducted twice)

🌐 Idempotency in HTTP

In REST APIs, some HTTP methods are defined as idempotent by standard.

According to **Internet Engineering Task Force** (which defines HTTP specs):

Method	Idempotent?	Why
GET	✅ Yes	Just fetches data
PUT	✅ Yes	Replaces resource
DELETE	✅ Yes	Deletes resource
POST	❌ No	Usually creates new resource
PATCH	❌ Usually No	Partial updates

🔍 Why PUT Is Idempotent

Example:

```
PUT /users/1
{
 "name": "Prajwal"
}
```

No matter how many times you send this request:

Final state:

```
{
 "name": "Prajwal"
}
```

Same result every time.

🔥 Why POST Is Not Idempotent

Example:

```
POST /orders
{
 "item": "Laptop"
}
```

If user retries 3 times:

You may create 3 orders.

Different result each time.

Not idempotent.

🚨 Why Idempotency Matters in Real Systems

Because networks are unreliable.

Requests can:

- Timeout
- Retry automatically
- Be duplicated

If your operation is not idempotent:
You can cause:

- Double payments
- Duplicate orders
- Data corruption

💳 Real-World Example: Payments

Payment APIs (like Stripe) use:

Idempotency keys

You send:

POST /charge
Idempotency-Key: 12345

Even if request is retried:
Server processes it only once.

This prevents double charging.

🧠 Idempotency Key Pattern

Example in Node:

```
const processed = new Set();

app.post("/pay", (req, res) => {
  const key = req.headers["idempotency-key"];

  if (processed.has(key)) {
    return res.send("Already processed");
  }

  processed.add(key);

  // Process payment
  res.send("Payment done");
});
```

Now repeated requests don’t duplicate action.

🎯 Idempotent ≠ Safe

Important distinction:

Concept	Meaning
Safe	Does not change server state (GET)
Idempotent	Can change state, but repeated calls don't change final result

🏦 Where Idempotency Is Critical

- Payment systems
- Order processing
- Distributed systems
- Microservices

- Retry mechanisms
- Message queues

⚠ Common Developer Mistake

They assume:

"Frontend won't send request twice."

Wrong.

Reality:

- User double clicks
- Mobile retries
- Proxy retries
- Load balancer retries

If backend isn’t idempotent → chaos.

JSON

JSON = **JavaScript Object Notation**

```
{
  "name": "Prajwal",
  "age": 22
}
```

Important:

- Keys must be in double quotes
- No functions
- No undefined
- No comments

🔥 Core JSON Methods in JavaScript

There are only **two main JSON methods** in JavaScript:

1️⃣ JSON.stringify()

Converts JavaScript object → JSON string.

Basic Example

```
const obj = { name: "Prajwal", age: 22 };

const json = JSON.stringify(obj);

console.log(json);
```

Output:

```
{"name":"Prajwal","age":22}
```

Now it's a string.

Type:

```
typeof json // "string"
```

Why We Use stringify()

- Send data over HTTP
- Store in localStorage
- Save in database
- Send via WebSocket

⚠ What stringify Removes

```
const obj = {
  name: "Prajwal",
```

```
  greet: function () {},
  value: undefined,
  symbol: Symbol("id")
};
```

After stringify:

{"name":"Prajwal"}

Removed:

- Functions
- undefined
- Symbols

1️⃣ Replacer (Filter properties)

```
const obj = {
  name: "Prajwal",
  age: 22,
  password: "secret"
};

const json = JSON.stringify(obj, ["name", "age"]);
console.log(json);
```

Output:

{"name":"Prajwal","age":22}

Password removed.

2️⃣ JSON.parse()

Converts JSON string → JavaScript object.

Basic Example

```
const json = '{"name":"Prajwal","age":22}';

const obj = JSON.parse(json);

console.log(obj.name);
```

Now it's usable JS object.

⚠ Important Rule

This fails:

```
JSON.parse("{ name: 'Prajwal' }");
```

Because:

- Keys must be double quoted
- Strings must be double quoted

Correct:

```
JSON.parse('{"name":"Prajwal"}');
```

🧠 JSON vs JS Object

JSON	JS Object
String format	Memory object
Only data	Can have methods
Strict syntax	Flexible